

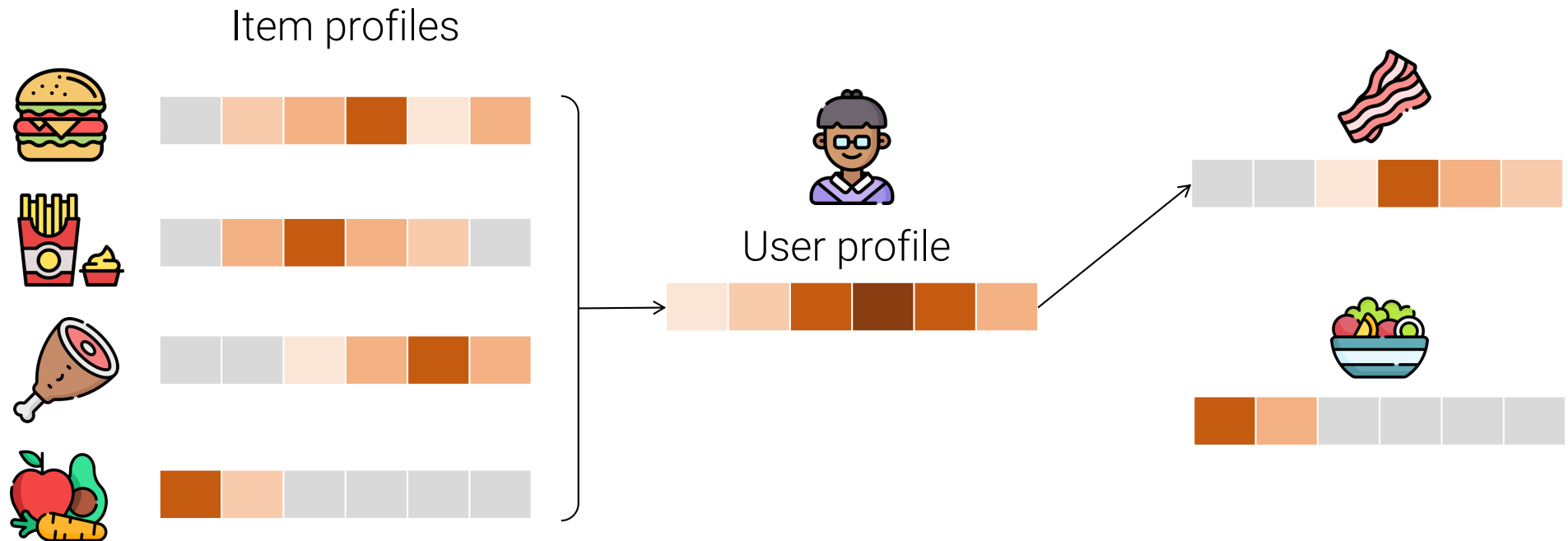
Content-based – Linear Regression approach

							 Items
Users	A 	4	2	3			
	B 	4			3		
	C 		1				5
	D 		5	3		4	5
Features	x_1	0.1	0.2	0.1	2.3	0.7	0.9
	x_2	0.3	0.5	0.1	1.2	0.7	1.6
	x_3	1.2	0.4	1.1	0.3	0.8	0.3

$$\hat{r} = \alpha^{(A)} + \beta_1^{(A)} x_1 + \beta_2^{(A)} x_2 + \beta_3^{(A)} x_3$$

$$\hat{r} = \alpha^{(D)} + \beta_1^{(D)} x_1 + \beta_2^{(D)} x_2 + \beta_3^{(D)} x_3$$

Content-based – Nearest Neighbors approach



- Extract profile (feature representation) for items
- Aggregate at user level for a user profile
- Find the a new item that matches the user profile more closely

Aggregated user representation

User rating (avg.=3) User normalized rating

	Action	Comedy	Crime	Drama	Noir	Horror	Romance	Sci-fi	War	Western
A		3.0		1.5					2.0	
B				1.1	2.2		1.5			
C		2.0		0.5			1.0		1.5	

5 = 5-3 = 2

1 = 1-3 = -2

3 = 3-3 = 0

i -th position of user u 's vector Number of rated items Normalized ratings
 $\downarrow$ $\nwarrow$ $\downarrow$

$$v_i(u) = \frac{1}{|\{v_i(k), v_i(k) > 0\}|} \cdot \sum_{k=1}^M v_i(k) \cdot r(u, k)$$
 $\uparrow$
 i -th position of items k 's vector

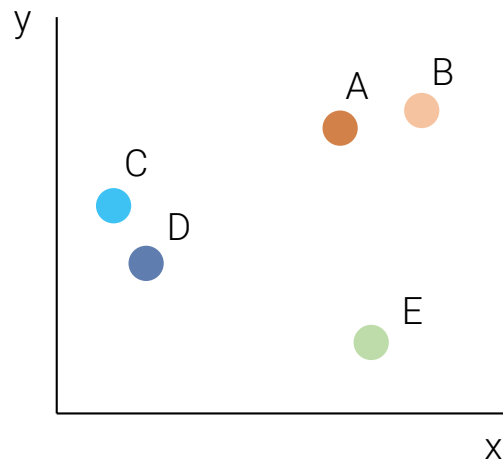
$$v_{drama}(u) = \frac{1}{3} \cdot \sum_{k=A,B,C} v_{drama}(k) \cdot r(u, k)$$

$$= \frac{1}{3} \cdot (1.5 \cdot 2 + 1.1 \cdot -2 + 0.5 \cdot 0) = 0.27$$

User profile

	1		0.27						1	
Action	Comedy	Crime	Drama	Noir	Horror	Romance	Sci-fi	War	Western	

Locality Sensitive Hashing (LSH)



	Hash
C 	1
E 	2
A 	3
D 	4
	5
B 	6

Traditional hashing

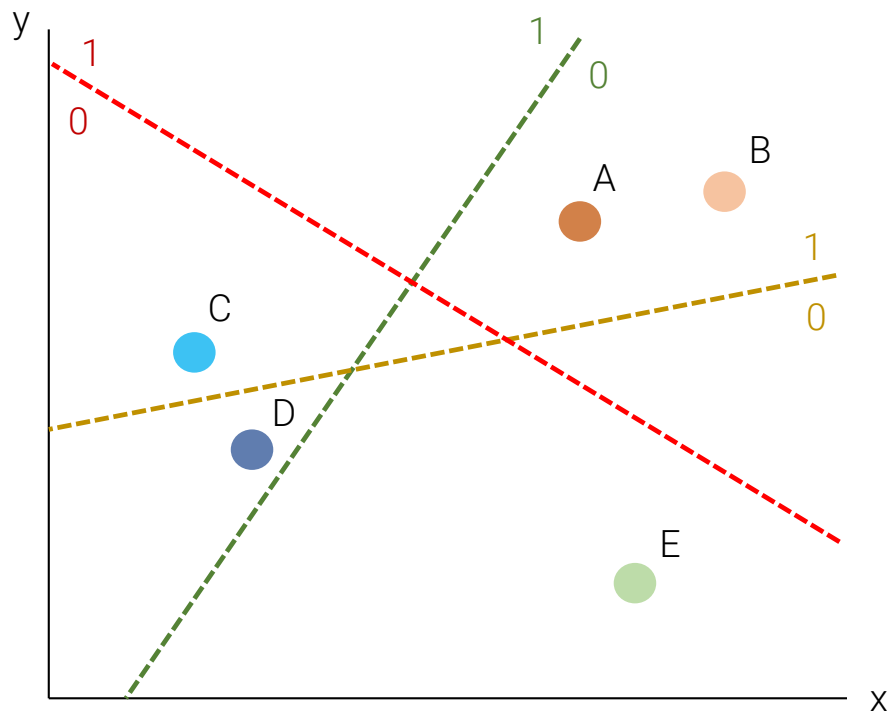
	Hash
E 	1
	2
A,B  	3
	4
C,D  	5
	6

Locality Sensitive Hashing

Family of functions that map data points to hash values such that points that are close in the feature space will have high probability of colliding in the hash space

Used to solve the problem of near-neighbor search

Locality Sensitive Hashing



In practice, hash values are obtained by the dot product between the vector and a random vector with the same dimension for which each entry is drawn by a Gaussian distribution

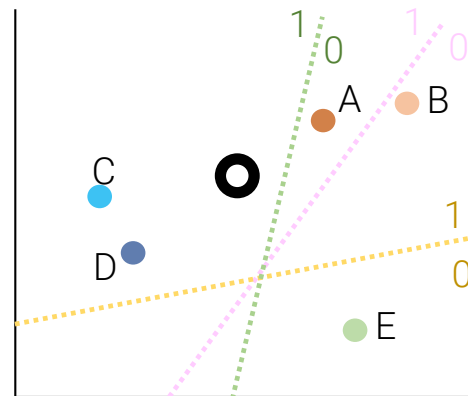
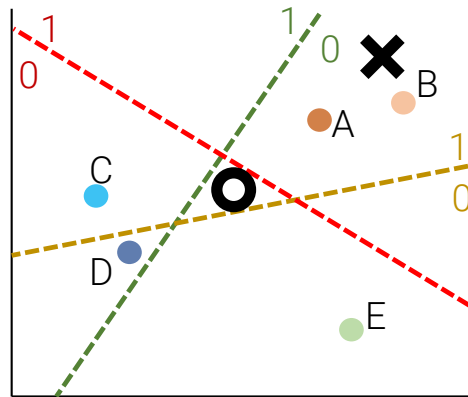
k, L : *amplification* parameters

A	B	C	D	E	}	k
						
1	1	0	0	0		
0	0	1	1	0		
1	1	1	0	0		
A	B	C	D	E	}	k
						
1	0	1	1	0		
1	1	1	1	0		
0	0	1	1	0		

L tables (bands, cohorts)

$$h(\vec{v}) = \begin{cases} 1 & \text{if } \vec{v} \cdot \vec{r} \geq 0 \\ 0 & \text{if } \vec{v} \cdot \vec{r} < 0 \end{cases}$$

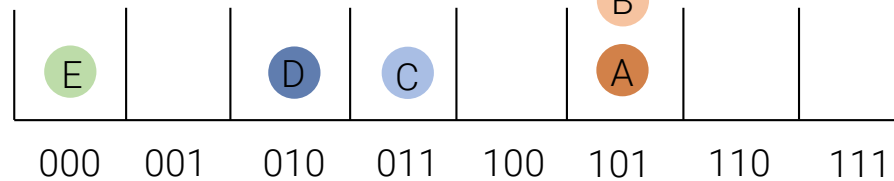
Locality Sensitive Hashing



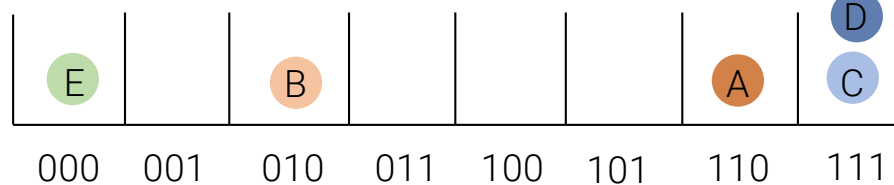
A match is found if there is a collision in at least one of the tables

○ No neighbors

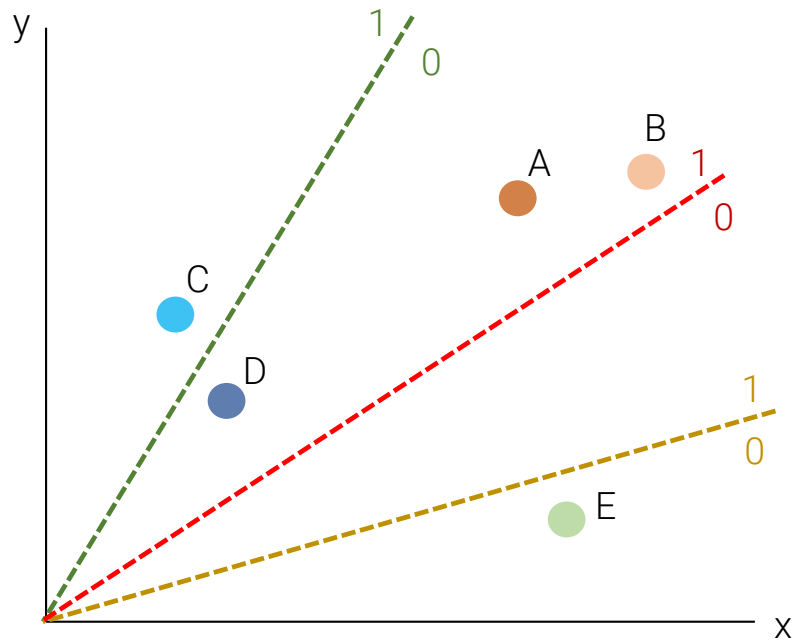
✗ Neighbors: [(B, dist1), (A, dist2)]



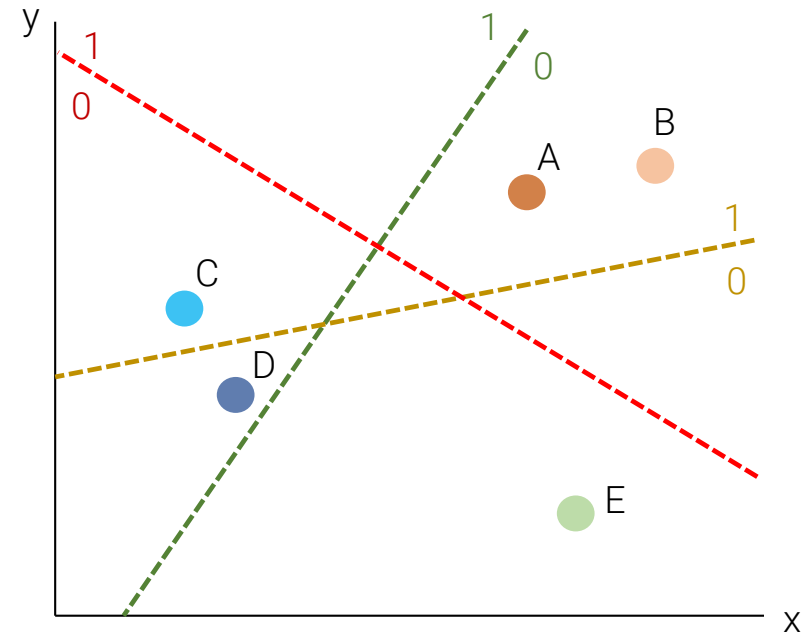
Neighbors: [(D, dist3), (C, dist4)]



Different Random Projections



Random hyperplanes centered on the origin
Buckets maximize cosine similarity



Random hyperplanes
Buckets maximize Euclidean distance

Sparse matrices in Python

```
from scipy.sparse import csr_matrix
# create sparse matrices from dense ones (other constructors available)
M1 = [[1,2,7],[0,2,1],[0,1,1],[0,0,3],[1,0,0]]
M2 = [[2,4,8],[1,1,1],[0,4,0],[0,1,0],[7,0,0]]
csr_M1 = csr_matrix(M1)
csr_M2 = csr_matrix(M2)
csr_M3 = csr_M1 + csr_M2 * 4 #execute common matrix operations

# stack vectors on top of each other
stacked_V = sparse.vstack([csr_matrix([1,2,3]),
                           csr_matrix([4,5,6])])
stacked_V.toarray() # convert to dense array

# get the number of nonzero entries on cols (axis=1 for rows)
csr_M1.getnnz(axis=0)
```


Sparse LSH in Python

```
from sparselsh import LSH

V = [x1,...,xn] #list of sparse vectors
V_info = [42,...,666] #list of information for each vector (e.g., vector ID)

# index vectors into the lsh. NUM_DIM is the size of the vocabulary
lsh = LSH(hash_size=6, input_dim=NUM_DIM, num_hashtables=2)
for v,i in zip(V,V_info):
    lsh.index(v, extra_data=i), axis=1)

# query the LSH, get the 5 closest results
query = V[0]
points = lsh.query(query, num_results=5)
```

<https://github.com/brandonrobertz/SparseLSH>

Pros and cons of content-based recommendations

- **Pros**

- No need for data on other users
- Handles the item cold start problem (works for new, unrated items)
- Able to recommend to users with unique taste
- Easy to explain the recommendations (I recommend X because you liked Y)

- **Cons**

- Hard to find the appropriate features
- Overspecialization (low serendipity)
- User cold start problem (user hasn't rated anything yet)

Neighborhood-based collaborative filtering

		Items					
							
Users		5	5	5			
		5	4		1	1	
		1			5	4	4
			3	4	4	4	5
					5	5	

Find neighborhoods of similar users or items (wrt ratings), and recommend ratings by aggregating ratings in the neighborhood

User-user (or user-based CF)

		Items					
							
Users		5	5	5		<u>1</u>	
		5	5	<u>5</u>		1	
		1	1		5	4	5
			5	4	4	4	1
					5	5	

Find users with similar rating profiles, recommend missing values based on other people's ratings

Item-item (or item-based CF)

		Items					
							
Users		5	5	5			
		5	5		<u>1</u>	1	
		1	1		5	4	5
			5	4	4	4	1
					5	5	

Find items with similar rating profiles, recommend missing values based on other items' ratings

User-user CF (global)

- Global average of all previous ratings by other users
- Same as non-personalized

$$\hat{r}_{ui} = \frac{\sum_{v \in U_i} r_{vi}}{|U_i|}$$

↑ Rating of item i by user v

↑ Set of users who rated item i

↑ Estimated rating of item i by user u

User u



$$\hat{r}_{ui} = 4.3$$



Item i



User-user CF (global, discounted)

- Scale user ratings by the individual user's average rating over all items they rated
- Offsets for different rating patterns

$$\hat{r}_{ui} = \bar{r}_u + \frac{\sum_{v \in U_i} (r_{vi} - \bar{r}_v)}{|U_i|}$$

Average rating given by user u
(or v) to previously-rated items

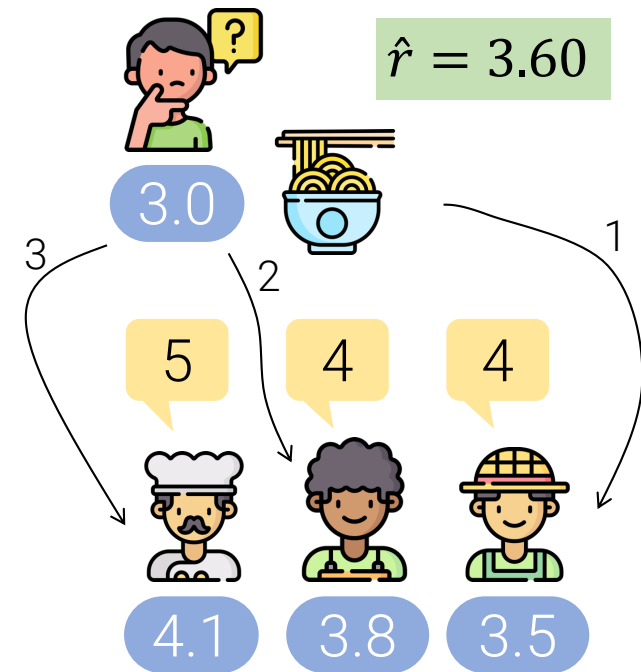


User-user CF (global, discounted, weighted)

- Consider ratings from some users more influential than others (e.g., because they are similar to the target user)

$$\hat{r}_{ui} = \bar{r}_u + \frac{\sum_{v \in U_i} (r_{vi} - \bar{r}_v) \cdot w_{uv}}{\sum_{v \in U_i} w_{uv}}$$

Weight that represents the strength of connection (or similarity) between users u and v

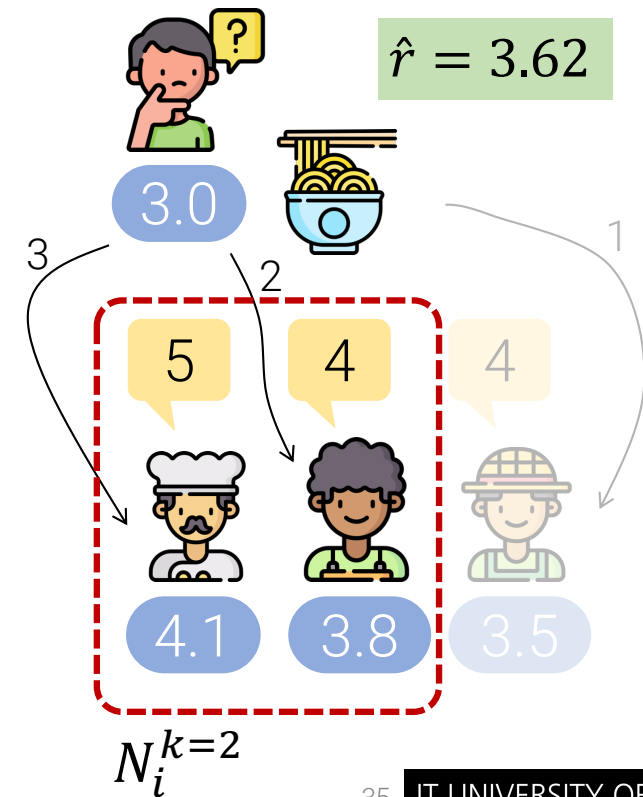


User-user CF (neighborhood, discounted, weighted)

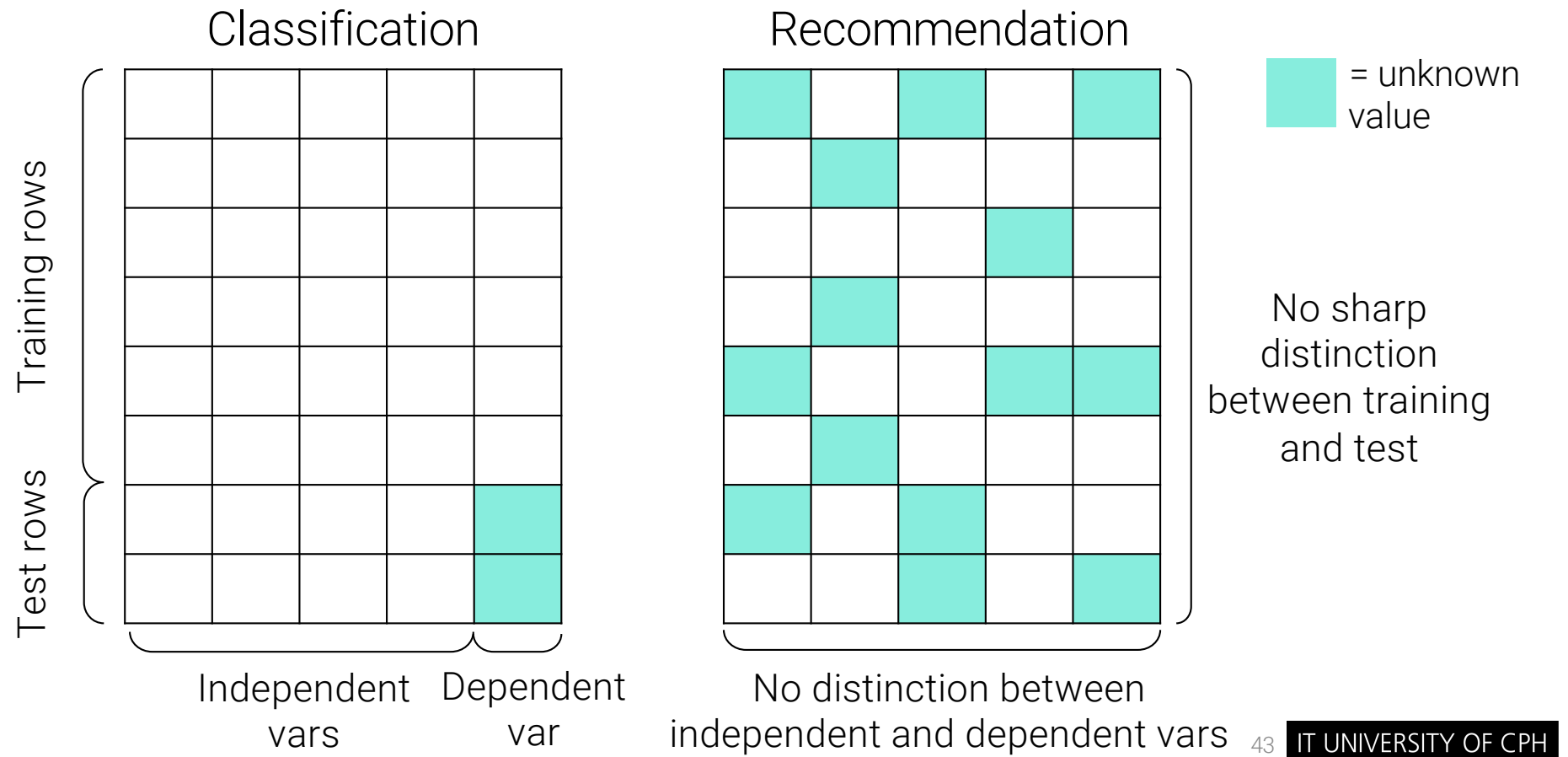
- Restricts the set of users to a neighborhood of people who are more similar to the target user

$$\hat{r}_{ui} = \bar{r}_u + \frac{\sum_{v \in N_i^k(u)} (r_{vi} - \bar{r}_v) \cdot w_{uv}}{\sum_{v \in N_i^k(u)} w_{uv}}$$

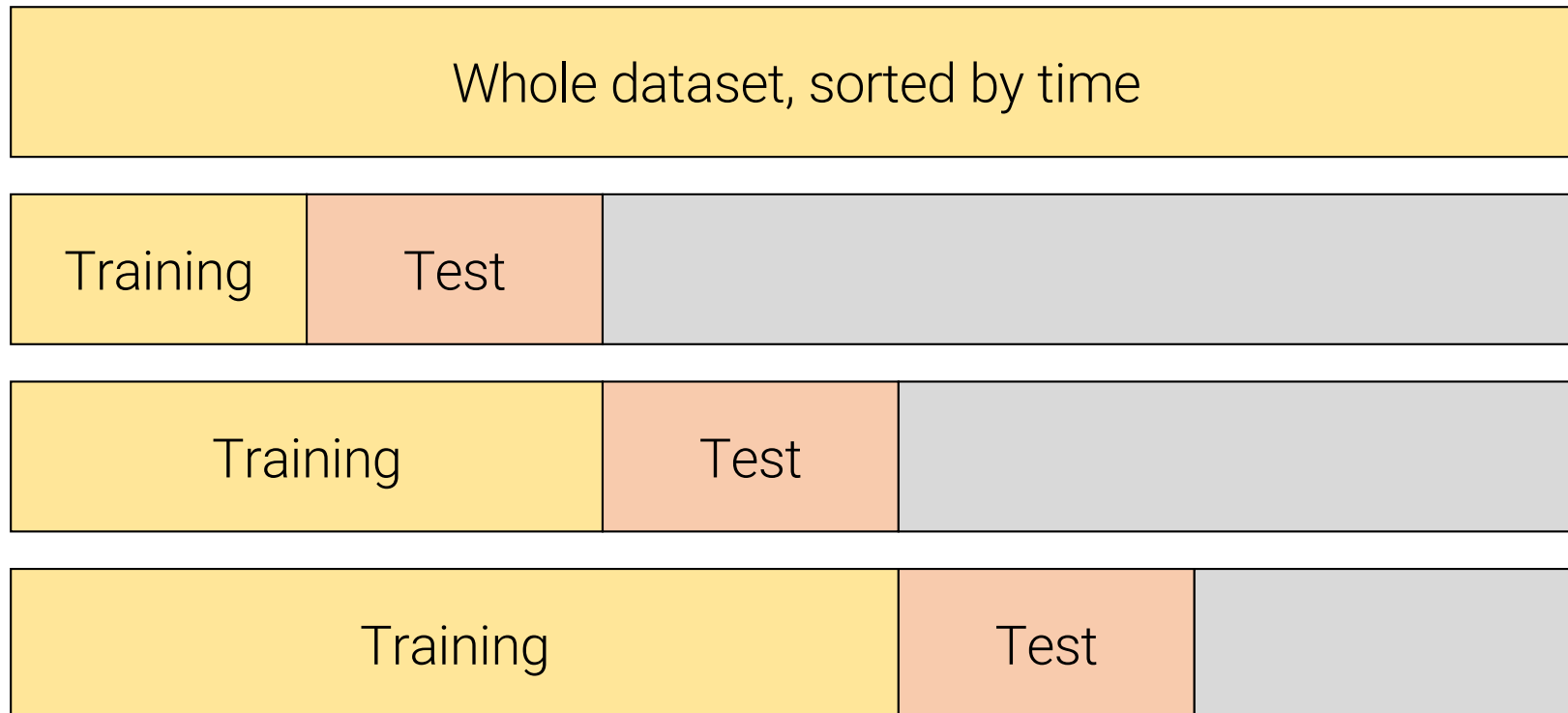
Set of top k neighbors of user u
who rated item i



Classification vs. Recommendation



Temporal evaluation



Neal Lathia "Evaluating Collaborative Filtering Over Time" 2010
<https://discovery.ucl.ac.uk/id/eprint/133957/1/133957.pdf>

Is evaluation based on past data enough?

- Recommender systems are different from machine learning tasks and general IR because they should **discover** new content, rather than **predict** correct answers
- On the right, an example of a ranking that is evaluated as good, but in practice is not great
- Online evaluation (controlled experiments) can measure the value of a recommender system more faithfully



Predicted rank	Would have rated:
1 	★★★★☆
2 	★★★☆☆
3 	★★★★★

Food.com recipes

	user_id	recipe_id	rating
0	2046	4684	5.0
1	2046	517	5.0
2	1773	7435	5.0
3	1773	278	4.0



Ratings

kaggle

Recipes

	recipe_id	name	minutes	n_steps	n_ingredients	calories	fat	sugar	sodium	protein	saturated	carbs
0	137739	arriba baked winter squash mexican style	55	11	7	51.5	0.0	13.0	0.0	2.0	0.0	4.0
1	31490	a bit different breakfast pizza	30	9	6	173.4	18.0	0.0	17.0	22.0	35.0	1.0
2	112140	all in the kitchen chili	130	6	13	269.8	22.0	32.0	48.0	39.0	27.0	5.0

<https://www.kaggle.com/shuyangli94/food-com-recipes-and-user-interactions>

Assignment 1: Food.com recipes content-based recs

- Build a content-based recommender system that uses linear regression to predict ratings
- Try it out on users with a high number of ratings
- Try some train-test split to evaluate performance

MovieLens



- GroupLens is a research group specialized in RecSys
- MovieLens is GroupLens online service that collects movie ratings and provides recommendations
- 27M ratings (with timestamps) of 58K movies (with publication year) by 280K users until year 2018

<https://grouplens.org/datasets/movielens/>

MovieLens

movieId		title	genres
0	1	Toy Story (1995)	Adventure Animation Children Comedy Fantasy
1	2	Jumanji (1995)	Adventure Children Fantasy
2	3	Grumpier Old Men (1995)	Comedy Romance
3	4	Waiting to Exhale (1995)	Comedy Drama Romance
4	5	Father of the Bride Part II (1995)	Comedy

Movies

userId	movieId	tag	timestamp
0	2	60756	funny
1	2	60756	Highly quotable
2	2	60756	will ferrell
3	2	89774	Boxing story
4	2	89774	MMA

Tags

userId	movieId	rating	timestamp
0	1	1	4.0
1	1	3	4.0
2	1	6	4.0
3	1	47	5.0
4	1	50	5.0

Ratings

Assignment 2: MovieLens content-based recs

- Build a content-based recommender system
 - Build the profile of a user to get a list of 10 candidate movies close to their taste (and different from the ones they see already, ofc)
 - The profile can be based on movie tags or movie genres, or both
 - Use a knn technique to predict the ratings that the user might give to those 10 recommended movies
- You can use the small version of the dataset for your first experiments, and then try it on the full dataset
 - (with my inefficient implementation, the whole notebook takes 30-40 mins to run)

Nearest-neighbor recommendation practice

- Step 1: Calculate item profiles
- Step 2: Index item profiles into LSH
- Step 3: Calculate user profiles
- Step 4:
 - Select a target user to provide recommendations to
 - Query LSH with the user vector get a rank of candidate items
- Step 5:
 - Create another LSH index containing the item vectors of the target user
 - Query the user LSH index with the vectors of each recommended movie to estimate the predicted rating based on k-nearest neighbors

rating	rating_scaled	title		genres
5.0	1.263158	Eternal Sunshine of the Spotless Mind (2004)	Drama	Romance Sci-Fi
4.5	0.763158	Lord of the Rings: The Two Towers, The (2002)	Adventure	Fantasy
4.5	0.763158	One Flew Over the Cuckoo's Nest (1975)	Drama	
4.5	0.763158	Fantasia (1940)	Animation Children	Fantasy Musical
4.5	0.763158	Monsters, Inc. (2001)	Adventure Animation Children	Comedy Fantasy
4.0	0.263158	Chasing Amy (1997)	Comedy Drama	Romance
4.0	0.263158	Dogma (1999)	Adventure Comedy	Fantasy
4.0	0.263158	Schindler's List (1993)	Drama	War
4.0	0.263158	Office Space (1999)	Comedy	Crime
3.5	-0.236842	Casper (1995)	Adventure	Children

Example of
content-
based
movie recs

User profile

rank	movieid	title		genres
2	134853	Inside Out (2015)	Adventure Animation Children	Comedy Drama Fantasy
3	160706	Der Nachtmahr (2015)	Drama	Mystery
4	48082	Science of Sleep, The (La science des rêves) (...)	Comedy Drama	Fantasy Romance
5	2692	Run Lola Run (Lola rennt) (1998)	Action	Crime
6	72356	Partly Cloudy (2009)	Animation Children	Comedy Fantasy
7	148367	Sanjay's Super Team (2015)	Animation	
8	148367	Sanjay's Super Team (2015)	Animation	
9	104644	Maria Bamford: The Special Special Special! (2...	Comedy	
10	6953	21 Grams (2003)	Crime Drama	Mystery Romance Thriller

Recommendations

Assignment 3: MovieLens CF



```
xcode-select --install # for mac users  
pip install scikit-surprise
```

- Train a User-user CF model to recommend movies from the MovieLens dataset
- Compute the RMSE across different values on K
- Use the Surprise recommendation library
- (requires a C compiler functioning and in your path)

https://surprise.readthedocs.io/en/stable/model_selection.html